商家后台与可观测性

BossID 数据基础 / merchant 角色路线图 / Jaeger / Skywalking / 静默降级

gomall · 业务侧 deck

今日大纲

商家数据现状：BossID 在、接口缺

三层鉴权边界：merchant 角色的路线图

Jaeger：链路追踪解决的真实问题

日志、降级与业务码

商家数据现状：BossID 在、接口缺

一个商家在 gomall 里能干什么、不能干什么

- **能干**：注册账号、上架商品、被买家下单、被买家收藏。
- **不能干**：查”我的今日营收”、查”我的待发货”、查”我的库存告警”、自助退款。
- 五张核心表里 **BossID 字段早就埋好了**：`order.BossID`、`product.BossID`、`cart.BossID`、`favorite.BossID`、`skill_product.BossID`。
- 但是 `/api/v1/merchant/*` **一个路由都没有**——数据有、查询入口缺。

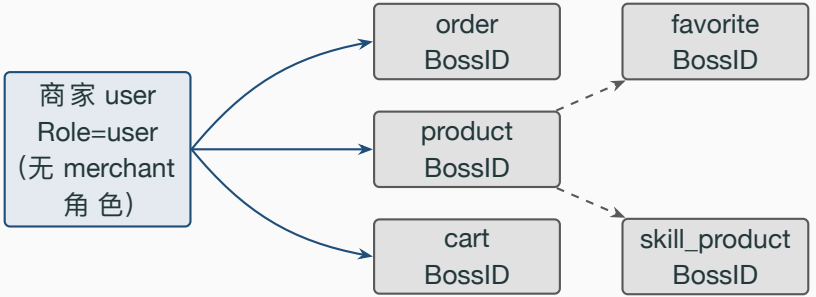
repository/db/model/order.go:12、product.go:23、cart.go:10、favorite.go:12 五处 BossID

现状对业务的真实伤害

- **招商场景**：BD 谈一个新品牌，对方第一句话” 我能看到自己卖了多少吗” → 答不上来。
- **客服场景**：商家把客服当后台用——” 帮我查下我店铺今日订单”、” 帮我看 SKU 12345 库存”。1 个客服对 N 个商家，工单全是低价值查询。
- **越权风险**：临时给商家开 admin 账号 → 商家立刻看到**所有店铺**数据，谈判桌上当场翻车。
- **数据正确但权限模型没成形**：`product/delete` 已经按 `WHERE id=? AND boss_id=?` 隔离，但没有任何中间件保证调用者就是 `boss_id` 这个人。

repository/db/dao/product.go:67–72 已带 boss_id 隔离；routes/router.go:148–155 admin 子组只有 3 个 handler

BossID 散落在五张表里



五个外键**物理上都在了**，只是**逻辑上没有 merchant API 把它们串起来**。补一个 merchant group 不需要改 schema，只是补 5–10 个 handler。*repository/db/model/order.go:12*、*product.go:23*、*cart.go:10*、*favorite.go:12*

数据基础够不够给商家看板

看板模块	需要的字段	现有	缺什么
今日营收	order(BossID, Money, Type=2)	✓	时间维度索引
待发货	order(BossID, Type=2, Ship?)	部分	无 ship_status 字段
库存告警	product(BossID, Num)	✓	阈值规则表
客诉工单	无 ticket 表	缺	整张表

4 个模块里

2 个开箱可做（今日营收、库存告警），1 个差一个字段（待发货），1 个要新加表（客诉）。**先做能做的**，是常见的最小可用商家后台路线。*repository/db/model/order.go:8–18 Type* 字段只有”已支付/未支付”，没有发货状态

三层鉴权边界：merchant 角色的路线图

今天只有两层墙，明天要加中间一层

角色	路径	现状	备注
user	`/api/v1/*`	✓	默认放行
merchant	`/api/v1/merchant/*`	**缺**	中间一层待建
admin	`/api/v1/admin/*`	部分	已有 3 个 handler

第三层 admin 已落地

`(RequireRole("admin"))`；**缺的是中间一层 merchant**。*routes/router.go:148–155 admin 子组；middleware/rbac.go:48–86 RequireRole*

merchant 角色落地需要改的四处

1. `repository/db/model/user.go:27--32`：新增常量 `RoleMerchant = "merchant"`。Role 字段已有，无需 migration。
2. `middleware/rbac.go`：`RequireRole` 已是变长参数 `(allowed ...string)`，调用方传 `"merchant","admin"` 即可。
3. `routes/router.go`：新增 `merchant := authed.Group("/merchant") + merchant.Use(middleware.RequireRole("merchant","admin"))`。
4. `service/admin.go`：补一个 `PromoteToMerchant`，参考 `PromoteToAdmin:46--62` 直接复刻；记得调 `InvalidateRoleCache`。

四处改动各 5–15 行，不需要任何 DB migration。整套路线图**两个 PR 就能合**。

routes/router.go:148–155 现成模板；*service/admin.go:46–62 promote* 逻辑可复用

RequireRole + lookupRole：30 秒缓存的取舍

Listing 1: *middleware/rbac.go:22–45 + :89–91*

```go
var (
   roleCache sync.Map
   roleCacheTTL = 30 * time.Second
)
func lookupRole(ctx context.Context, userId uint) (string, error) {
   if v, ok := roleCache.Load(userId); ok {
      e := v.(roleCacheEntry)
      if time.Now().Before(e.expires) { return e.role, nil }
   }
   u, err := dao.NewUserDao(ctx).GetUserById(userId)
   if err != nil { return "", err }
   role := u.Role
   if role == "" { role = "user" }
   roleCache.Store(userId, roleCacheEntry{role: role,
      expires: time.Now().Add(roleCacheTTL)})
   return role, nil
}
func InvalidateRoleCache(id uint) { roleCache.Delete(id) }
```

lookupRole 逐行讲解

- **为什么要缓存**：每个 admin / merchant 请求都进 `RequireRole`，不缓存就每次打 DB；高并发下 user 表的 `SELECT WHERE id=?` 会变成路由瓶颈。
- **为什么是 30s**：商家提权 / 降级是低频事件，30s 内权限漂移可接受；同时单点回源 DB 的 QPS 被砍到 1/30。
- **为什么用 sync.Map**：单实例进程内只读为主、写少；不引入 Redis 避免把鉴权链路绑到 Redis 健康度上。
- **为什么提权后必须显式 Invalidate**：商家刚被审核通过，第一次访问 `/merchant/*` 不能等 30s。`service/admin.go:59` 调 `InvalidateRoleCache(targetUserId)` 做精确失效。

service/admin.go:59 PromoteToAdmin 末尾的显式失效

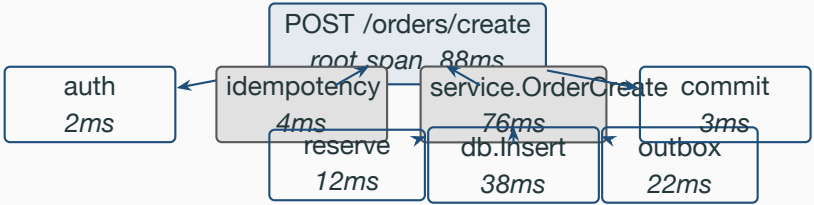
Jaeger：链路追踪解决的真实问题

为什么要在 router 第一行就挂 Jaeger

- gomall 一次下单要穿过：`ratelimit` → `Jaeger` → `Cors` → `Auth` → `Idempotency` → `OrderCreate` → `Redis Lua` → `MySQL` → `Outbox`。
- 出问题时**只看错误日志**永远定位不到瓶颈：哪一段慢、哪一段挂、哪一段触发了重试——日志里只有”成功”或”失败”。
- Jaeger 把这条链做成一棵 **span 树**：每段耗时、每次依赖调用、每个上下游 traceId 都能串起来。
- 业务价值：客服报”用户 X 下单 5 秒才返回” → 用 trace 一找 → 看到 4.7s 卡在第三方支付的 `paydown` 步。

routes/router.go:23 全局挂 `middleware.Jaeger()`

一次下单的 trace span 树



一棵树就把” 慢在哪一段” 答清楚。**root 88ms 中 db.Insert 占 38ms**，是该接口下一步优化的着力点。

service/order.go OrderCreate 主路径；track.WithSpan 在子函数里开 child span

Jaeger middleware：traceId 透传

Listing 2: *middleware/track.go:11–29*

```go
11 func Jaeger() gin.HandlerFunc {
12    return func(c *gin.Context) {
13       traceId := c.GetHeader("uber-trace-id")
14       var span opentracing.Span
15       if traceId != "" {
16          var err error
17          span, err = track.GetParentSpan(
18             c.FullPath(), traceId, c.Request.Header)
19          if err != nil { return }
20       } else {
21          span = track.StartSpan(
22             opentracing.GlobalTracer(), c.FullPath())
23       }
24       defer span.Finish()
25       c.Set(consts.SpanCTX, opentracing.ContextWithSpan(c, span))
26       c.Next()
27    }
28 }
```

Jaeger middleware 逐行讲解

- **第 3 行**：读 `uber-trace-id` header；格式 `<traceId>:<spanId>:<parentId>:<flags>`。
- **第 5–9 行**：有 traceId → 当前请求是别人调过来的，开 **child span**，root 留在上游。
- **第 10–13 行**：没 traceId → 当前请求是入口，自己开 root span。
- **第 14 行**：`defer span.Finish()` 必须有，否则 Jaeger 端会看到”未结束”灰色 trace。
- **第 15 行**：把 span 写回 `c.Context`，下游通过 `ctl.GetSpanContext` 开 child span ——**这一步就是”链路”的串接**。

pkg/utils/track/track.go:61–78 GetParentSpan Extract

Jaeger 的开销：实测低于 5%

场景	RPS	p95
`/ping` 裸基线（含 ratelimit/cors/Jaeger 全中间件）	64,254	3.51ms
`/product/show` 真业务（MySQL PK + 全中间件）	62,226	3.01ms

- `/ping` 已经过了 `Jaeger()`（router.go:23 注册），不是” 无 trace 裸 RPS”。
- 商品详情 RPS 仅比 ping 低 **3.2%**，证明全链路 trace 在 const sampler 全量采样下，单机损耗 $<$ 5%。
- 真把损耗摁住，还能从 `const` 切到 `probabilistic` 抽样（默认 0.001%），CPU 占用再降一个数量级。

stressTest/REPORT.md 链路压测表第 2、3 行；pkg/utils/track/track.go:15–27 GetDefaultConfig const sampler

Jaeger trace 在业务排障的三种典型用法

症状	用法	找的是什么
单笔卡 5s	按 traceId 拉单条	哪一段 span 最长
某接口变慢	按 service+op 拉 p99	p99 是哪段 span 抬起来的
报错 70002	按 status 过滤	CB Open 前最后一次成功调用是何时

关键是

把 traceId 写进业务日志：客服 / 商家投诉里只要附上 traceId，SRE 就能 1 分钟内定位。当前 gomall 的 `log.LogrusObj` 默认 JSON formatter 已经能塞字段，只差在 handler 层把 traceId 写进去。*pkg/utils/log/logger.go:29–31 JSON formatter；middleware/track.go:26 span 写回 ctx*

为什么 cmd/main.go 顶上有一行神秘 import

- `cmd/main.go:7` 写着 `_ "github.com/apache/skywalking-go"`。
- 这是 **Skywalking Go agent 的接入方式**：纯 import side-effect，不调任何 API。
- 编译时配合 `skywalking-go-agent` 工具链**重写字节码**（Go 没有 JVM agent，靠 source-level rewrite），自动把标准库 / gin / gorm 的调用包成 span 上报。
- `docker-compose.yml:101--148` 已经把 skywalking-oap + skywalking-ui 拉起来了，监听 11800（agent 上报）/ 8080（UI）。

cmd/main.go:7 import；docker-compose.yml:101–148 skywalking-oap/ui

Jaeger 与 Skywalking 同时存在的取舍

维度	Jaeger（OpenTracing）	Skywalking
插桩方式	业务代码显式开 span	agent 自动织入
学习曲线	中（要懂 carrier/inject）	低（接入即用）
定制空间	高（可任意标 tag）	中（agent 决定）
UI / 拓扑	单服务 trace 视角	全链路服务拓扑图
gomall 用法	已落地，主线	备选，docker 起着

业务选型逻辑：单服务调

试用 Jaeger 够细；多服务上线后看拓扑用 Skywalking。两者不冲突，可以并存——当前 gomall 把两套基础设施都备齐，主推 Jaeger，留 Skywalking 做 SRE 大盘。*cmd/main.go:44*

注释掉的 `//track.InitJaeger()` 留了切换开关

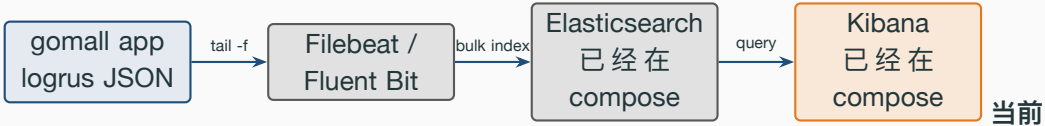
日志、降级与业务码

logrus 已经把 JSON 字段对齐了，差最后一公里

- `pkg/utils/log/logger.go:29--31` 已经设置 `JSONFormatter` + 标准时间格式。
- 全站调用 `log.LogrusObj` 的地方共 **349 处**，覆盖 middleware / service / repository 三层。
- 输出目标：`logs/<date>.log`，本地文件 append。
- **差什么**：日志没带 traceId / userId / bossId，工单里” 用户 X 7:23 报错” 没法快速捞到对应日志。
- **差什么 (2)**：日志只落文件，没集中收集；多实例部署后排障要 ssh 一台台抓。

pkg/utils/log/logger.go:14–41 InitLog；grep -rn log.LogrusObj 共 349 处

ELK / EFK 路线图：三步走



完成度：左两格还没接、右两格 `docker-compose.yml:44--99` 已经起着 ES 8.4.2 + Kibana。
下一步只差：选一个 collector（Filebeat 或 Fluent Bit），把 `logs/*.log` tail 到 ES 索引。

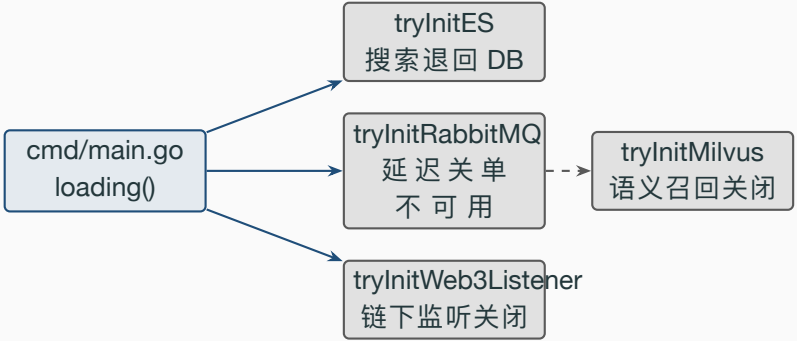
docker-compose.yml:44–71 elasticsearch；:85–99 kibana；pkg/utils/log/logger.go:38 留了 ES hook 接入位

traceId 接入日志的最小改动

- 当前 `log.LogrusObj.Errorln("rbac lookup role failed:", err)` 这样的调用很多。
- 改造成 `log.LogrusObj.WithFields(logrus.Fields{"traceId": tid, "userId": uid}).Errorln(...)`。
- traceId 从 `ctl.GetSpanContext(c)` 取——这个工具函数已经在 `pkg/utils/ctl/ctl.go:73` 写好了。
- 改动量：包一层 `log.WithCtx(ctx)` 的 helper，全站逐步替换。
- 收益：客服把 traceId 贴进搜索框 → 全链路日志 + 全链路 trace 同时召回。

pkg/utils/ctl/ctl.go:73 `ctx.Get(consts.SpanCTX)` *取出 span 上下文*

gomall 启动期的四个 tryInit



- 四个外部依赖任意一个挂了，HTTP 主路径**照常起**：登录、浏览、下单、支付都不受影响。
- 用户最多失去某一项**增量能力**：搜不到语义结果、订单不自动延迟关闭，但不会”白屏 500”。

cmd/main.go:37–42 调用；*:54–97* 四个 *tryInit* 实现

tryInitES：8 行 recover 套路

Listing 3: *cmd/main.go:64–73*

```go
// tryInitES ES 不可用时静默跳过，搜索接口退回 DB 路径
func tryInitES(ctx context.Context) {
   defer func() {
      if r := recover(); r != nil {
         util.LogrusObj.Warnf("ES 初始化失败，搜索退化：%v", r)
      }
   }()
   es.InitEs()
   initialize.InitSearch(ctx)
}
```

tryInitES 逐行讲解：为什么不传 error

- **第 3–8 行 defer + recover**：底层 `es.InitEs()` 是**老代码**，连不上 ES 直接 `panic`。改 `InitEs` 签名风险大（连锁影响 search service），所以**在外层包一个 recover**。
- **为什么不返回 error 让 main 决定**：main 拿到 error 也只能写日志接着启——业务上"ES 挂" 和" 启动失败" 是**两件事**，前者不应阻塞 HTTP 启动。
- **为什么用 Warnf 不是 Errorf**：error 级别会触发告警；这里是" 已知降级路径"，不是异常。给 SRE 留 Warn 让他主动看，不要凌晨 3 点叫人起床。
- **相同套路用了 4 次**：`tryInitRabbitMQ`、`tryInitES`、`tryInitWeb3Listener`、`tryInitMilvus` ——同一种姿势，业务上是**对" 不可控外部依赖" 的统一回应**。

cmd/main.go:54–97 四个 trylnit 同构

静默降级 vs 显式降级：什么时候用哪个

依赖类型	降级策略	gomall 实现
启动期初始化（ES/Milvus/RMQ）	静默 recover	cmd/main.go:54–97
请求期慢依赖（第三方支付）	熔断器	middleware/circuitbreaker.go
请求期限流（秒杀）	拒绝 + 业务码	70001
请求期幂等命中	直接返回	60002

- **启动期**：不影响主流程，静默；启动一次就过去了。
- **请求期**：每次都触发，必须显式让客户端知道”现在不行，待会再来”。
- 两套思路并行，不矛盾——启动静默 + 请求显式，是 gomall 现在的统一姿态。

docs/slides/10-business-traffic-governance.tex 配套讲请求期降级

现有业务码段已经分好域

段	含义	现有码	备注
20xxx	业务通用错误	ERROR=20001 等	兜底
30xxx	鉴权 / token	ErrorAuth*	rbac 在用
50xxx	第三方 / 存储	ErrorOss=50001	oss 上传
60xxx	并发 / 幂等	60002 幂等命中	idempotency
70xxx	流量治理	70001/70002	limit/cb

各段已经清晰隔开。

merchant 自助接口需要新增 80xxx 段（暂定），覆盖” 店铺未审核 / 资质过期 / 不属于本店 SKU / 提现额度不足” 等。*pkg/e/code.go* 现有错误码段

建议的 80xxx 段业务码（待设计）

码	含义	客服话术
80001	店铺未通过资质审核	” 您的店铺还在审核，预计 1–3 个工作日”
80002	当前操作不属于本店 SKU	” 该商品不属于您的店铺”
80003	提现额度不足	” 本月提现额度已满，下月 1 号刷新”
80004	待发货超时即将自动改判	” 您有 N 单超过 72 小时未发货”
80005	库存低于商家自设阈值	”SKU XXX 库存 < 阈值，请尽快补货”

编

码原则同 deck 10：**HTTP 200 + status 字段**，app / 商家 PC 端按 status 分支显示文案；不暴露内部技术细节，但要让客服**足够区分**。*docs/slides/10-business-traffic-governance.tex* 业务码-话术映射模板

三类人共用一套底层信号

角色	关心	看什么
商家	卖了多少、待发几单	商家后台（待建）
SRE	p99、错误率、依赖健康度	Jaeger / Skywalking
客服	单个 traceId 怎么挂的	日志 + trace 联查

三个视角共用**同一套信号**

（JSON 日志 + trace span 树），差别只在聚合维度。gomall 现状：信号已采集，**聚合视图还没搭**。下一步两个 PR：merchant 角色 + 看板 API；再下一个迭代：traceId 入日志 + Filebeat 接入。

> 可观测性是为了让**业务方独立解释自己的现状**——商家解释自己卖得好不好，SRE 解释系统稳不稳，客服解释这一单为什么挂。

面试 Q&A

Q1：BossID 都在了，为什么不直接给商家开 admin?

A：admin 看全站数据，商家只能看自店；权限边界不能用” 先开放再监控” 的姿态做。merchant 角色 + RequireRole(”merchant”) 才是正确解。

Q2：Jaeger 在 64K RPS 下还撑得住吗?

A：实测 `/product/show` 全中间件跑 62,226 RPS（`/ping` 64,254），开销 < 5%。const 全量采样能压得住；担心损耗可切 probabilistic 0.001%。

Q3：tryInitES 为什么用 recover 不传 error?

A：底层 `es.InitEs()` 老代码 panic 风格；改签名牵连大。外层 recover + Warnf 是**对” 启动期不可控依赖” 的统一回应**，与请求期熔断（70002）形成两套互补。

Q&A（续）

Q4：roleCache 为什么 30s 不是 5min?

A：商家被提权后等 5min 才生效，谈判桌上没法解释。30s 是” 权限漂移可接受 + 回源 DB 砍 30 倍” 的平衡点；提权时显式 `InvalidateRoleCache` 让**当事人立即生效**。

Q5：Jaeger 和 Skywalking 同时存在，是不是重复建设?

A：单服务调试 Jaeger 细，多服务拓扑 Skywalking 直观。两者数据模型相同（trace + span），并存只是双写不冲突；当 gomall 真上微服务后，Skywalking 的全局拓扑图是 SRE 的主视角。

Q6：商家自助接口和 admin 接口，业务码段为什么要分?

A：商家看到的错误码会出现在**商家 App 文案**里；admin 看到的只在内部后台。两套话术系统不同，业务码隔离段能避免文案串号。

代码位置一览

BossID 散落字段 *model/order.go:12, product.go:23, cart.go:10, favorite.go:12*

RBAC 中间件 *middleware/rbac.go:22–45, :48–86, :89–91*

Jaeger middleware *middleware/track.go:11–29*

Trace utility *pkg/utils/track/track.go:15–78*

Skywalking agent *cmd/main.go:7 import* 副作用

logrus 初始化 *pkg/utils/log/logger.go:14–72*

静默降级 *cmd/main.go:54–97* 四个 *tryInitX*

admin promote 模板 *service/admin.go:46–62*

admin 路由组 *routes/router.go:148–155*

压测锚点 *stressTest/REPORT.md ping/product/show* 表

配套：18-merchant-business.tex（商家旅程）/ 10-business-traffic-governance.tex（流量治理）